

Sliding Puzzle Project — Development Log

Day 1 — Sunday, July 26, 2026

Goal for the project

Build four Python puzzle projects (a playable sliding tile puzzle, a solver for it, a playable 3D Rubik's cube, and a solver for that) as a learning exercise, working through each one with guidance rather than having them built directly.

Today's focus

Getting a working development environment set up before writing any puzzle code, using VS Code as the editor.

What happened

- Decided to start with Project 1: a playable 2D sliding puzzle (3x3, 3x4, or 4x4) using pygame.
- Laid out a staged build plan: (0) environment setup, (1) board data model, (2) core movement logic, (3) shuffling, (4) rendering, (5) input handling, (6) game feel (timer/move counter/win detection), (7) supporting multiple board sizes.
- Hit a chain of environment issues before any puzzle code could be written:
 - pip and python were not recognized in Command Prompt — no system Python was installed yet, despite Python being usable inside Thonny and VS Code already (Thonny bundles its own interpreter).
 - Chose to install Python via python.org rather than the Microsoft Store, to get a normal pip/python on PATH.
 - First attempt used python.org's new "Python Install Manager" download, which doesn't have the classic "Add python.exe to PATH" checkbox and isn't what we wanted — cancelled it.
 - Installed the classic standalone installer for Python 3.14.6 instead, with "Add python.exe to PATH" checked. `python --version` / `pip --version` then worked correctly in Command Prompt.
 - Running `pip install pygame` failed: pygame tried to build from source (no prebuilt wheel yet for Python 3.14) and the build step failed with an SSL certificate error while fetching an SDL2 dependency. Root cause (confirmed via search): pygame's build system currently depends on `distutils.msvccompiler`, which Python 3.14 removed — a known, widespread compatibility issue, not specific to this machine.
 - Fix: uninstalled Python 3.14, installed Python 3.12.10 instead (the last Python 3.12 release with a full Windows installer). `pip install pygame` then succeeded immediately using a prebuilt wheel (pygame-2.6.1).
 - In VS Code, the Python extension (already installed) wasn't showing the "Python: Select Interpreter" command at all. Root cause: the workspace was open in Restricted Mode (VS Code's Workspace Trust feature), which disables/limits extensions until the folder is explicitly trusted. Trusting the workspace fixed it immediately — the command appeared, and dozens of other Python commands were now available.
- Selected the Python 3.12 (64-bit) interpreter (recommended, from the Python312 folder) in VS Code, ignoring an unrelated Python313 interpreter that was also listed.

- Created a throwaway test.py with “import pygame; print(pygame.ver)”, ran it in VS Code, and confirmed pygame 2.6.1 loads correctly.
- Connected a permanent project folder (“python puzzle project”, synced via OneDrive) so the dev log and all future code have a real, backed-up home.

Concepts & Lessons Learned (study notes)

PATH (environment variable)

PATH is a list of folder locations the operating system searches, in order, whenever you type a command name (like python) without giving its full file location. If the folder holding python.exe isn't on that list, the terminal can't find it — hence “not recognized as an internal or external command.” The installer's “Add python.exe to PATH” checkbox appends Python's install folder to that list so every terminal (Command Prompt, PowerShell, VS Code's terminal) can find python and pip without typing the full path each time.

Interpreters vs. the code you write

An “interpreter” is the actual program that reads and executes your Python code — python.exe itself. You can have several installed side by side (Thonny's bundled one, plus separate 3.14 and 3.12 installs today), and each keeps its own separate collection of installed packages. Installing pygame into one interpreter does not make it available to a different one — which is why “which interpreter is selected” in an editor matters: it decides which python.exe runs your file and which pip installs packages for it.

Packages, pip, and PyPI

pip is Python's package manager: it downloads and installs reusable libraries from PyPI (the Python Package Index), a public repository of open-source packages. “pip install pygame” means: look up pygame on PyPI and fetch it for this interpreter.

Prebuilt wheels vs. building from source

Libraries that include compiled C/C++ code (pygame wraps a graphics library called SDL2) usually publish ready-made binary files called “wheels” for each combination of operating system, CPU, and Python version. If a matching wheel exists, pip just downloads and drops it in — fast, nothing to compile. If none exists yet (as with pygame on the very new Python 3.14), pip falls back to compiling the package from source on your own machine, which needs the right build tools and can fail in ways a prebuilt install never would. That's exactly what the SSL/SDL2 error was — a failed source build, not a broken download.

Bleeding-edge vs. stable software versions

Brand-new major language releases (Python 3.14 came out very recently) are usually ahead of the surrounding ecosystem of third-party libraries, which need time to catch up and confirm compatibility. General lesson: unless you specifically need a brand-new feature, prefer a stable, slightly older release with a mature, well-tested ecosystem over the newest version — it avoids exactly this kind of debugging detour.

Sandboxing / trust models (VS Code's Restricted Mode)

Many developer tools limit what code or extensions are allowed to do by default, until you explicitly grant trust — protecting you from opening someone else's project and having it silently run scripts on your machine. This is the same “least privilege by default, then explicit trust” pattern you see in browser permissions or mobile app permissions. VS Code's Workspace Trust / Restricted Mode is that pattern applied to a code editor — untrusted folders get extensions disabled or limited until you say “I trust this.”

Status at end of day

Environment is fully working — Python 3.12.10, pygame 2.6.1, VS Code with a trusted workspace and the correct interpreter selected. No puzzle code written yet; that starts next.

Next up

Build the board data model for the sliding puzzle (Stage 1) — representing the board, tracking the blank tile, and printing it to the console — before touching any graphics.